

HEAVYSKILL: Heavy Thinking as the Inner Skill in Agentic Harness

Jianing Wang¹ Linsen Guo¹ Zhengyu Chen¹ Qi Guo^{1,2} Hongyu Zang¹ Wenjie Shi¹ Haoxiang Ma¹
Xiangyu Xi¹ Xiaoyu Li¹ Wei Wang¹ Xunliang Cai¹

Abstract

Recent advances in agentic harness with orchestration frameworks that coordinate multiple agents with memory, skills, and tool use have achieved remarkable success in complex reasoning tasks. However, the underlying mechanism that truly drives performance remains obscured behind intricate system designs. In this paper, we propose **HEAVYSKILL**¹, a perspective that views heavy thinking not only as a minimal execution unit in orchestration harness but also as an inner skill internalized within the model’s parameters that drives the orchestrator to solve complex tasks. We identify this skill as a two-stage pipeline, i.e., parallel reasoning then summarization, which can operate beneath any agentic harness. We present a systematic empirical study of HEAVYSKILL across diverse domains. Our results show that this inner skill consistently outperforms traditional Best-of-N (BoN) strategies; notably, stronger LLMs can even approach Pass@N performance. Crucially, we demonstrate that the depth and width of heavy thinking, as a learnable skill, can be further scaled via reinforcement learning, offering a promising path toward self-evolving LLMs that internalize complex reasoning without relying on brittle orchestration layers.

1. Introduction

Recently, large language model (LLM) agents have demonstrated remarkable success on solving complex reasoning tasks via an orchestrated harness (Meng et al., 2026; Wang et al., 2024b), reinforcement learning with verified rewards (RLVR) (Guo et al., 2025; Zheng et al., 2025a), and self-evolving learning (Gao et al., 2026; Wang et al., 2026; 2025b). To better guide the LLM agent to perform task

execution, Claude Code (Claude) develops *skills library* to inject extended knowledge and reusable strategies to the model with optional RLVR (Xu & Yan, 2026). Inspired by this technique, multiple flexible harnesses have been proposed, such as CodeX (Chen et al., 2021), Claude Code (Claude), OpenClaw (OpenClaw, 2024), and Hermes (Hermes-Agent, 2024). Under this harness, the LLM acts as multiple different agents, and performs complex tasks through an orchestrator and accompanying Skill and Memory components. However, the underlying mechanism that truly drives performance remains obscured behind intricate system designs.

Looking back at common harness models, orchestrator models typically operate within an agent loop, activating multiple subagents to execute various tasks in parallel based on user instructions and planning protocols, and ultimately summarizing the results. We believe this mode can be simplified into a two-stage workflow of *parallel thinking* and *summarization*. In a word, **we abstract the agentic harness into the LLM’s inherent capability of heavy thinking**. This approach revealed that the pursuit of empowering LLMs’ reasoning capability focused on extensive parallel reasoning, a test-time scaling (TTS) strategy that amplifies computational resources during the inference phase. This success underscores a fundamental insight: LLMs can substantially benefit from exploring multiple reasoning trajectories before converging to a final answer, mirroring the cognitive process of human collective deliberation. Recent efforts on parallel reasoning have primarily relied on specialized architectural modifications, reasoning pattern design, and large-scale post-training recipes (Pan et al., 2025; Liu et al., 2024; Jin et al.; Rodionov et al., 2025; Hsu et al., 2025; Yang et al., 2025b;c; Zheng et al., 2025b; Wen et al., 2025). Specifically, these methods (Hsu et al., 2025; Zheng et al., 2025b; Wen et al., 2025) modify the existing thinking pattern with multiple inline thinking tags to elicit the LLMs to derive multiple trajectories simultaneously, following a summary stage to aggregate the different rationales to the final answer. In contrast, alternative frameworks such as Kimi K2 (Bai et al., 2025), and PaCoRe (StepFun-AI, 2025), and LongCat-Flash-Thinking-2601 (Wang et al., 2026) have demonstrated promising results by decomposing heavy thinking into two distinct stages: a parallel reasoning stage provides some

¹Meituan LongCat Team ²National Engineering Research Center for Software Engineering, Peking University, Beijing, China. Correspondence to: Jianing Wang <lygwjn@gmail.com>.

Preprint. May 5, 2026.

¹<https://github.com/wjn1996/HeavySkill>.

independent reasoning trajectories, followed by a sequential deliberation stage that aggregates all trajectories and outputs a final answer.

In this paper, we conduct a systematic empirical investigation of heavy thinking skill for orchestrated harness, and propose HEAVYSKILL to consolidate the insights into a readable skill for LLM. We first provide a simple but effective training-free framework that decomposes the heavy thinking into two separate phases, i.e., parallel reasoning, and sequential deliberation. In this framework, we also introduce a memory cache mechanism to store and organize reasoning trajectories, enabling iterative deliberation where the model progressively refines its reasoning by revisiting and synthesizing prior attempts. Through extensive experiments spanning STEM, coding, and general tasks, we observe that heavy thinking substantially outperforms single reasoning. On STEM-oriented benchmarks with verifiable numerical answers, we also show a consistent performance hierarchy: Heavy-Pass@k $\geq$ Heavy-Mean@K $\geq$ Vote@K $\geq$ Mean@k. Notably, models with stronger intrinsic reasoning abilities can approach Pass@k upper bounds under heavy thinking, suggesting that sequential deliberation enables effective identification and synthesis of correct reasoning paths. Qualitative analysis reveals that models explicitly compare trajectory differences during deliberation, functioning as implicit verifiers.

Further analysis investigates which components substantially contribute towards the final performance. Ablation studies illuminate the complementary roles of framework components. We find that the quality and diversity of trajectories generated from parallel reasoning stage are two keys for the performance. We also show that sequential deliberation almost relies on the general capability of the model employed in this stage, suggesting that separate optimization of thinking and deliberation models may yield additional gains. In addition, we demonstrate that reinforcement learning from verifiable rewards (RLVR) can be adapted to optimize both reasoning breadth (via parallel generation) and depth (via deliberation), simultaneously improving Heavy-Mean@k and Pass@k metrics.

The main contributions are shown in the following. This work makes three primary contributions: 1) We introduce a simple but effective training-free framework for reproducing heavy thinking through parallel-reasoning and sequential deliberation. 2) We are the first to conduct the comprehensive empirical study to exhibit the performance of heavy thinking across diverse model scales and task domains, establishing its effectiveness and limitations. 3) We provide systematic analyses and insights into the interplay between framework components, and demonstrate the potential of heavy-mode-aware reinforcement learning as a superior optimization paradigm for reasoning-centric LLMs.

2. Methodology

2.1. Workflow of Heavy Thinking

We thus describe the framework of heavy thinking. The overview of architecture is shown in Figure 1. The inference pipeline is decomposed into two separate phases, including parallel reasoning, and sequential deliberation.

Given a problem q , the goal of the parallel reasoning phase is to produce multiple independent trajectories. Formally, we can obtain $\mathcal{T}_{\pi_\theta}(q, K) = \{y_1, \dots, y_K\}$, where K denotes the number of trajectories, π_θ represents the LLM that aims to produce these trajectories, $y_i = \{\pi_\theta(y_{ij}|q, y_{i,<j})\}_{j=1}$ is one of the generated trajectories.

When the parallel reasoning is finished, we choose another LLM π_ϕ to produce a summary content in the sequential deliberation, which can be viewed as a second-time reasoning process that aggregates these trajectories derived from π_θ . Formally, we can obtain $\mathcal{T}_{\pi_\phi}(x_c, K^{(1)})$, where $x_c = \mathcal{C}(\mathcal{T}_{\pi_\theta}(q, K))$ denotes the serialized memory cache derived from parallel reasoning, $K^{(1)}$ represents the number of generated summary content. We will describe this cache in the following section.

2.2. Serialized Memory Cache

To seamlessly bridge the two phases, we introduce a memory cache mechanism, which is a serialized context to store the candidate trajectories generated from the framework in history. Since each trajectory generated by reasoning models typically encompasses both extensive internal thinking content and answer content, serializing all complete trajectories would exceed the model’s maximum length limit. To ensure the robustness of subsequent inference, pruned trajectories are shuffled to prevent the model from developing a bias toward specific positions in the prompt. To this end, we define the serialized context as $\mathcal{C}(x_c)$, establishing it as the input for the sequential deliberation stage. The specific prompting function is shown in Appendix C.

2.3. Iterative Deliberation

We also introduce iterative deliberation, inspired by human behaviors in the real-world that repeatedly refine the ideas that were previously considered. Specifically, at the $t \in \{2, \dots, N\}$ iteration, we modify the memory cache by concatenating a loop input from the previous content from the sequential deliberation, i.e., $\mathcal{C}(x^{(t)})$, where $x_c^{(t)} = \mathcal{T}_{\pi_\phi}(x_c^{(t-1)}, K^{(t-1)}) || x^{(t-1)}$ is the modified cache, $K^{(t-1)}$ is the number of generated summary content phase, $||$ is the concatenation operation, N is the total number of the iteration.

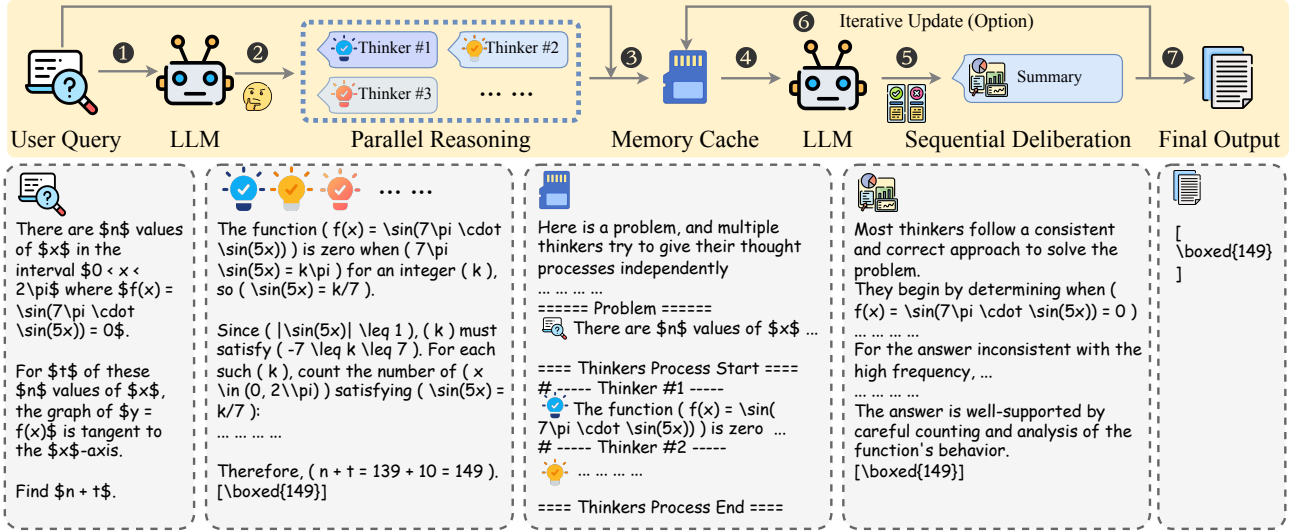


Figure 1. The overview framework of heavy thinking in LLMs test time scaling.

2.4. Readable Skill for Agentic Harness

This workflow provides a concrete Python pipeline for executing heavy thinking. However, modern agentic harnesses—such as Claude Code (Claude), CodeX (Chen et al., 2021), and Hermes (Hermes-Agent, 2024)—organize capabilities as *skills*: human-readable, model-interpretable documents that the orchestrator loads into its context window at inference time. A skill specifies *when* to activate, *how* to execute, and *what* to output, without requiring any code modification to the harness itself. This motivates us to distill the heavy thinking workflow into a single readable skill file.

Skill Structure. A readable skill is a structured natural-language document that serves as an executable specification for the LLM orchestrator. The HEAVYSKILL document consists of four components:

- **Activation Conditions** A declarative description of when heavy thinking should be triggered. The skill instructs the orchestrator to activate when facing tasks that involve complex reasoning and to remain dormant for simple factual queries or casual conversation. This conditional activation ensures that the additional inference cost is only incurred when the task complexity justifies it.
- **Parallel Reasoning Protocol** Instructions for the orchestrator to spawn K independent reasoning agents in parallel, each solving the same problem from scratch without access to other agents’ outputs. The skill encourages diversity by suggesting that agents employ different problem-solving strategies (e.g., algebraic versus geometric approaches). In the harness context, each agent corresponds to a subagent call, which is natively supported by modern orchestration frameworks.

- **Deliberation Prompt** The core of the skill is a carefully designed prompt template for the sequential deliberation stage. This prompt, which corresponds to the “General-Prompt” in our workflow implementation, instructs the deliberation model π_ϕ to: 1) *classify the query type* to determine the appropriate analysis depth; 2) *critically evaluate* each thinker’s reasoning rather than naively following the majority; 3) *re-derive* the answer when all thinkers are judged to be incorrect; and 4) *maintain language and format consistency* with the original query. The prompt explicitly prohibits superficial concatenation of thinker outputs and instead demands genuine synthesis. The full prompt is presented in Figure 7 (Appendix C).
- **Output Constraints** The skill specifies that the final response must contain only the answer—not the meta-analysis—and must follow the format conventions of the target domain (e.g., `\boxed{\cdot}` for mathematics, code blocks for programming).

From Workflow to Skill The key distinction between the workflow (Section 2.1) and the readable skill lies in the locus of control. In the workflow mode, an external Python pipeline orchestrates API calls, manages the memory cache, and routes outputs between stages. In the skill mode, the LLM orchestrator *itself* reads the skill document and autonomously executes the prescribed protocol—spawning parallel agents, collecting their outputs into its context window as a serialized memory cache, and performing deliberation in a subsequent generation step. This self-orchestration is made possible by the in-context learning capability of frontier LLMs, which can faithfully follow multi-step procedural instructions embedded in their prompt.

Portability and Generality Because the readable skill is a plain-text document with no framework-specific dependencies, it can be injected into any harness that supports skill loading and subagent spawning. We have verified that the same HEAVYSKILL document functions correctly under both Claude Code and custom orchestration harnesses, without modification. This portability aligns with our central thesis: heavy thinking is not an artifact of a particular system design but an *inner skill* that can be activated across diverse orchestration environments. By encapsulating the two-stage pipeline as a transferable skill, we decouple the reasoning capability from the infrastructure, enabling any sufficiently capable LLM to perform heavy thinking.

3. Experiments

3.1. Setups

By default, both phases of the LLM use the same model (i.e., $\pi_\theta = \pi_\phi$) unless otherwise specified. We choose multiple close-weight and open-weight models for evaluation. Concretely, the close-weights models consist of GPT-5-Thinking (OpenAI, 2025b), Claude 4.5 Thinking, and Gemini 3 Pro Preview. The open-weights models contain R1-Distill-Qwen-7B (Guo et al., 2025), R1-Distill-Qwen-32B (Guo et al., 2025), R1-Distill-Qwen3-8B (Guo et al., 2025), Qwen3-8B (Yang et al., 2025a), Qwen3-32B (Yang et al., 2025a), DeepSeek R1-0528 (Guo et al., 2025), GPT-OSS-20B (OpenAI, 2025a), Kimi K2 Thinking (Bai et al., 2025), GLM4.6 (Zeng et al., 2025), and DeepSeek V3.2 Thinking (DeepSeek-AI et al., 2025).

In the main experiments, we set temperature as 1.0, topp as 0.95 and topk=10. The number of iterations is $N = 1$, the number of parallel trajectories $K \in \{8, 16\}$, and the number of generated summary content is $K^{(1)} = 4$. For the metrics, we choose three basic values: 1) Mean@K (M@K) denotes the average accuracy of the selected K parallel trajectories from the parallel reasoning phase; 2) Pass@K (P@K) represents the proportion of the K selected trajectories where at least one is correct, which can be used to measures the boundary of the model’s inference ability. 3) Vote@K (V@K) denotes the accuracy of the trajectories with the highest frequency of answers, which is similar to BoN. We also design two metrics: 1) Heavy-Mean@K (HM@K) denotes the average accuracy of the content after the second phase; 2) Heavy-Pass@K represents the proportion of the generated summary contents where at least one is correct.

3.2. Evaluations on STEM Tasks

In this section, we evaluate the effectiveness of the “heavy thinking” framework across a wide range of STEM tasks, including AIME25, BeyondAIME, HMMT25-Feb, and

GPQA-Diamond. We compare HM@4 and HP@4 metrics against standard test-time scaling metrics, such as the mean performance (M@K), the intrinsic potential (P@K), and Majority Voting (V@K). The main results are shown in Table 1.

Heavy thinking consistently outperforms single-trajectory attempts

Our empirical results demonstrate that HM@4 consistently surpasses M@K across all models and STEM benchmarks. This indicates that parallel reasoning combined with sequential deliberation invariably yields a positive performance gain over the average quality of individual reasoning trajectories. Notably, when employing large-scale frontier models (e.g., Kimi K2 Thinking, GPT-5-Thinking), the heavy thinking often facilitates near-perfect scores on several benchmarks. These results are consistent with recent technical reports suggesting that scaling test-time compute through deliberation is a robust path toward saturation on difficult reasoning tasks.

Validation of Test-Time Scaling Laws By scaling the number of parallel trajectories (K) and employing sequential deliberation, we observe that the model’s performance does not merely plateau but continues to improve, effectively leveraging the increased inference budget. This confirms that heavy thinking serves as a practical realization of Test-Time Scaling, where the “width” of reasoning (parallel exploration) and the “depth” of deliberation (sequential synthesis) act as multipliers for the base model’s capability. This scaling property is particularly crucial for complex reasoning tasks where a single inference pass is often insufficient, validating that allocating more compute at test time is a reliable strategy for boosting performance without retraining.

Superiority over heuristic voting strategies As highlighted by the blue cells in Table 1, the performance of heavy thinking frequently exceeds that of the heuristic Majority Voting (V@K) strategy. This suggests that sequential deliberation is more effective at synthesizing and distilling the results of parallel reasoning paths than simple statistical consensus. Interestingly, we observed that while highly capable models (e.g., DeepSeek R1-0528 and GLM-4.6) sometimes show parity with or slight underperformance compared to voting on AIME25, this is primarily due to a ceiling effect—these models already achieve exceptional scores (above 90), leaving minimal room for further differentiation. However, on more cognitively demanding benchmarks such as BeyondAIME, HMMT, and GPQA-Diamond, the advantage of the heavy thinking over voting becomes significantly more pronounced, underscoring its utility for complex problem-solving.

HEAVYSKILL: Heavy Thinking as the Inner Skill in Agentic Harness

Models	k	AIME25					BeyondAIME					HMMT25-Feb					GPQA-Diamond				
		M@ k	P@ k	V@ k	HM@4	HP@4	M@ k	P@ k	V@ k	HM@4	HP@4	M@ k	P@ k	V@ k	HM@4	HP@4	M@ k	P@ k	V@ k	HM@4	HP@4
Leading Close-Weights Models																					
GPT-5 Thinking	8	92.5	100	96.7	96.7	96.7	69.9	86.0	67.0	79.5	83.0	90.4	96.7	93.3	93.1	96.7	85.8	96.5	89.9	89.9	91.7
	16	91.9	100	96.7	99.2	100	70.1	91.0	73.0	82.5	88.0	89.8	96.7	86.7	95.0	96.7	85.6	97.5	86.4	87.2	90.9
Claude 4.5 Thinking	8	82.5	90.0	90.0	90.0	90.0	58.9	74.0	66.0	63.3	67.5	66.3	76.7	66.7	75.8	76.7	82.1	91.9	80.3	83.1	86.9
	16	83.1	93.3	90.0	90.0	90.0	59.4	83.4	70.0	68.3	71.5	69.4	86.7	80.0	85.8	86.7	81.5	94.9	78.8	81.7	85.9
Gemini-3 Pro Preview	8	95.0	96.7	96.7	95.8	96.7	83.1	96.0	83.0	92.0	95.0	95.4	100	100	100	100	-	-	-	-	-
	16	94.8	96.7	96.7	95.8	96.7	-	-	-	-	-	96.5	100	93.3	100	100	-	-	-	-	-
Open-Weights Models																					
R1-Distill-Qwen-7B	8	42.1	66.7	50.0	50.0	56.7	26.5	54.0	32.0	30.8	39.0	32.1	56.7	43.3	32.3	37.3	48.8	85.9	49.5	51.5	63.1
	16	41.7	66.7	60.0	56.7	60.0	28.1	59.0	36.0	35.3	45.0	29.2	80.0	40.0	31.7	43.3	49.0	89.9	51.1	51.8	65.7
R1-Distill-Qwen-32B	8	53.3	76.7	63.3	63.3	66.7	30.3	57.0	40.0	41.0	47.0	21.3	46.7	33.3	39.2	43.3	64.5	88.4	66.7	66.9	71.7
	16	52.3	80.0	66.7	68.3	76.7	31.4	59.0	46.0	44.3	49.0	23.3	63.3	43.3	45.8	50.0	64.3	88.4	66.7	67.2	71.7
R1-Distill-Qwen3-8B	8	76.7	90.0	83.3	85.8	90.0	54.1	70.0	60.0	59.0	65.0	58.3	80.0	66.7	65.0	66.7	61.2	85.4	62.1	64.8	72.2
	16	73.3	86.6	80.0	80.8	83.3	52.8	76.0	58.0	56.5	61.0	57.9	86.6	60.0	68.3	73.3	61.5	90.4	66.7	68.1	74.2
Qwen3-8B	8	69.6	86.7	76.7	80.0	80.0	46.1	66.0	53.0	52.5	58.0	47.9	76.7	56.7	56.7	56.7	59.2	80.3	62.1	63.3	66.2
	16	70.0	86.7	80.0	80.8	83.3	45.6	72.0	53.0	52.3	56.0	49.0	83.3	56.7	58.3	60.0	-	-	-	-	-
Qwen3-32B	8	72.5	83.3	83.3	80.8	83.3	52.0	70.0	58.0	58.5	62.0	58.8	83.3	73.3	60.8	70.0	69.8	85.4	70.7	71.7	76.3
	16	73.1	90.0	80.0	86.7	86.7	51.9	77.0	58.0	58.5	65.0	57.9	90.0	60.0	62.5	70.0	69.0	88.4	69.7	70.3	76.3
DeepSeek R1-0528	8	87.1	96.7	90.0	93.3	93.3	67.3	84.0	73.0	74.0	77.0	80.8	93.3	86.7	91.7	93.3	80.6	93.4	82.3	84.6	87.9
	16	87.3	96.7	90	96.7	96.7	-	-	-	-	-	78.5	93.3	83.3	91.7	93.3	80.2	94.4	81.3	83.3	85.9
GPT-OSS-20B	8	92.2	96.7	96.7	92.5	96.7	65.0	82.0	69.0	67.5	74.0	78.3	90.0	90.0	83.3	93.3	-	-	-	-	-
	16	92.0	96.7	96.7	93.3	96.7	-	-	-	-	-	76.1	93.3	90.0	85.0	90.0	-	-	-	-	-
Kimi K2 Thinking	8	95.4	100	96.7	100	100	76.8	87.0	81.0	83.0	84.0	87.5	100	90.0	93.3	93.3	85.2	94.9	82.3	86.9	89.9
	16	95.2	100	96.7	99.2	100	-	-	-	-	-	88.5	100	93.3	90.0	93.3	85.3	97.0	80.3	87.5	91.4
GLM 4.6	8	91.3	96.7	96.7	96.7	96.7	74.1	90.0	78.0	78.8	81.0	91.3	100	96.7	100	100	82.9	93.9	79.8	85.2	88.9
	16	93.1	100	100	96.7	96.7	-	-	-	-	-	90.4	100	96.7	99.2	100	82.5	96.0	80.3	87.5	91.4
DeepSeek V3.2 Thinking	8	93.3	100	93.3	93.3	96.7	-	-	-	-	-	91.3	100	96.7	96.7	96.7	-	-	-	-	-
	16	93.5	100	96.7	100	100	-	-	-	-	-	92.3	100	93.3	100	100	-	-	-	-	-

Table 1. Overview performance of heavy mode on STEM tasks (Heavy Mean@4, simplify as HM@4) compared to basic TTS metrics in the parallel reasoning phase, i.e., Mean@K (M@K), Pass@K (P@K), and Vote@K (V@K).

Potential to surpass intrinsic reasoning boundaries

While it remains challenging for the aggregate performance (HM@4) to surpass the theoretical potential of the raw trajectories (P@K), our results show that HM@4 frequently approaches P@K in frontier models like DeepSeek V3.2 and GPT-5 Thinking. Remarkably, with a sufficiently LLM in the deliberation process, the potential of the heavy thinking (HP@4) exceeds the raw thinking potential (P@K) in nearly half of our experimental trials. This suggests that the deliberation process does not merely select from existing answers but can synthesize cross-trajectory insights to generate correct solutions that were not present in any single raw reasoning path. This finding provides a strong empirical

foundation for leveraging RLVR to further bridge the gap between HM@4 and HP@4, potentially pushing the limits of LLM reasoning beyond their inherent per-trajectory constraints.

3.3. Evaluations on General Reasoning Tasks

Task-Dependent Efficacy of Sequential Deliberation

Unlike the consistent gains observed in STEM tasks, the impact of the summary mechanism (HM@4) varies across general reasoning categories. On objective, verifiable tasks such as LiveCodeBench and IFEval, heavy thinking demonstrates substantial improvements. For instance, GPT-OSS-20B sees its performance surge from an M@K of 69.7% to an HM@4

Models	LiveCodeBench (24.08-25.05)				Arena-Hard				IFEval				IMO (Answer Bench)			
	M@k	P@k	HM@4	HP@4	M@k	P@k	HM@4	HP@4	M@k	P@k	HM@4	HP@4	M@k	P@k	HM@4	HP@4
Qwen3-8B	55.5	67.4	56.8	63.8	26.0	-	25.0	-	85.4	-	85.2	92.4	50.2	68.8	50.3	63.3
R1-Distill-Qwen3-8B	56.3	70.9	56.8	67.4	20.8	-	18.7	-	35.7	-	69.3	86.5	47.0	72.5	47.2	62.3
GLM 4.6	81.0	90.3	81.3	87.9	88.2	-	88.1	-	88.8	-	88.5	94.8	74.5	89.5	75.1	86.0
Kimi K2 Thinking	81.2	91.0	83.7	80.4	83.5	-	83.1	-	92.5	-	92.0	97.6	69.1	85.3	77.2	88.0
GPT-OSS-20B	69.7	89.0	69.2	85.5	25.4	-	25.0	-	90.8	-	91.1	97.6	65.8	81.5	71.0	84.5

Table 2. Overview performance of heavy thinking on general reasoning tasks (Heavy Mean@4, simplify as HM@4) compared to basic TTS metrics in the parallel reasoning phase, i.e., Mean@K (M@K) and Pass@K (P@K).

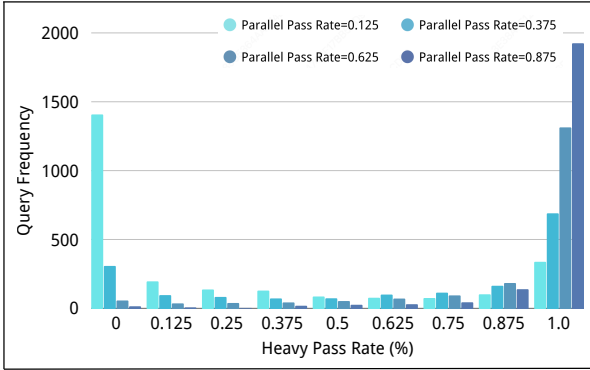


Figure 2. The pass rate distribution of heavy thinking in different pass rates of parallel reasoning.

of 85.5% on LiveCodeBench. Similarly, R1-Distill-Qwen-32B experiences a significant boost on IFEval (35.7% $\rightarrow$ 69.3%). This confirms that for tasks with clear logical or programmatic constraints, the summary model effectively distills high-quality solutions from multiple reasoning paths.

Challenges in Subjective Alignment On Arena-Hard, which focuses on human-like chat and open-ended preferences, the gains from HM@4 are more marginal or occasionally slightly negative. This suggests that while sequential deliberation excels at "correctness-oriented" tasks, its benefit is less pronounced in "preference-oriented" tasks where the "mean" of multiple responses may not necessarily align with the specific stylistic nuances favored by the reward model or judge.

Superiority of the Summary Potential A key finding in Table 2 is that the potential of the summary model (HP@4) consistently remains the highest metric across nearly all benchmarks. Notably, in tasks like IMO (Answer Bench), several models achieve HP@4;P@K (e.g., GLM 4.6 reach-

ing 86.0% vs. 75.1%). This indicates that the deliberation process is not merely selecting a winner from existing paths but has the capacity to "re-reason" and uncover correct answers that were initially missed in the raw P@K sampling.

4. Further Analysis

4.1. Can Sequential Deliberation Revises Parallel Thinking?

Our preliminary observations in Table 1 indicate that heavy thinking consistently outperforms vanilla majority voting, suggesting that the model possesses the intrinsic capability to discern and select correct answers even when they appear as low-frequency trajectories in parallel sampling. To further investigate this capability, we provide a granular analysis of the distributional relationship between the pass rates of parallel reasoning and heavy thinking.

Specifically, we choose open-resource data from Skywork OR1, DAPO, and DeepScaler, and leverage R1-Distill-Qwen-7B model as our experimental backbone. We randomly sample 10k queries and conduct parallel reasoning with a sampling size of $K = 16$ for each query to determine its baseline parallel pass rate. We then categorize queries into distinct groups based on specific parallel pass rate intervals $\{0.125, 0.375, 0.625, 0.875\}$. For each group, we construct a corresponding memory cache and perform sequential deliberation (without iteration), the number of generated responses $K^{(1)}$ is 16.

Results in Figure 2 illustrate the distribution of the heavy pass rate across different parallel pass rate cohorts. Our analysis yields the following key insights: 1) For queries with a parallel pass rate below 0.5, which typically pose a significant challenge for heuristic voting methods, heavy thinking demonstrates substantial corrective potential. Although approximately 1,400 queries remain unresolved, over 500

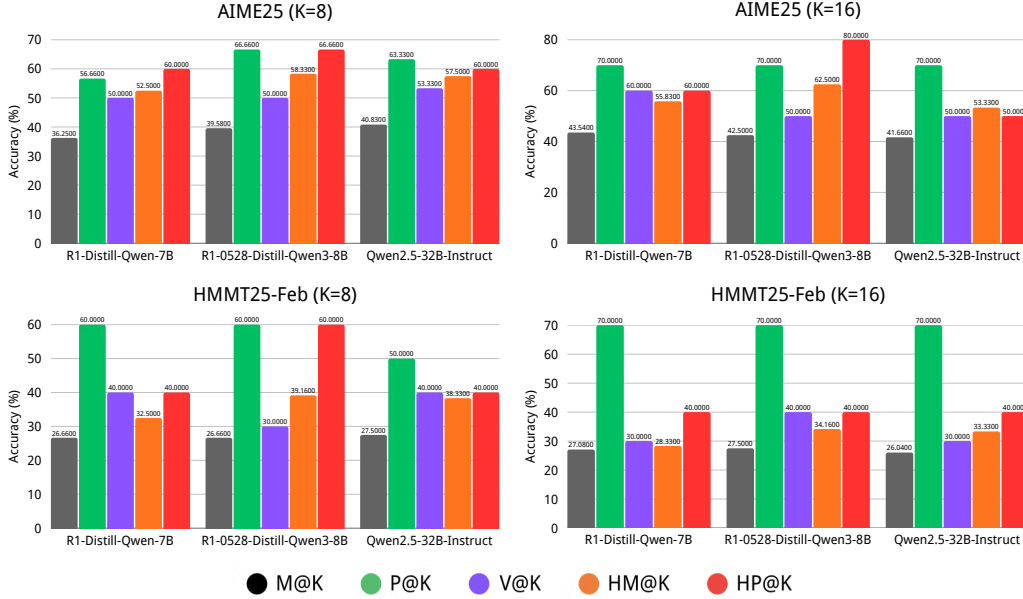


Figure 3. When fixing the LLM as R1-Distill-Qwen-7B in the parallel reasoning phase, the final performance of different LLMs in sequential deliberation.

queries are successfully rectified through deliberation. This empirical evidence underscores the model’s ability to refine errors even when the initial success probability is low. 2) In scenarios where the parallel pass rate exceeds 0.5, it maintains high accuracy, with a success rate exceeding 98%. While several queries (approximately 30) experience performance degradation, this loss is negligible compared to the overall performance gains achieved across the dataset.

4.2. Model Selection for the Role of Summary

To evaluate the robustness and generalizability of the framework, we investigate the performance when different models are paired for the two-stage reasoning process. Specifically, we fix the model for the parallel reasoning phase as R1-Distill-Qwen-7B. For the sequential deliberation stage, we select three models with varying architectures and scales: R1-Distill-Qwen-7B, R1-Distill-Qwen3-8B, and Qwen2.5-32B-Instruct.

The experimental results are illustrated in Figure 3. We observe that regardless of the model employed in the second stage, the HM@K metric consistently outperforms the baseline M@K across all tested benchmarks (AIME25 and HMMT25-Feb). This empirical evidence suggests that the heavy thinking framework is highly versatile and compatible with diverse model architectures, effectively enhancing reasoning performance through cross-model collaboration.

Furthermore, we highlight a counter-intuitive finding regarding the model choice for the second stage. While Qwen2.5-32B-Instruct itself does not exhibit superior per-

formance in solving these complex reasoning problems independently (Yang et al., 2025a)², its integration into the sequential deliberation phase yields results that align with our expectations of performance gains. This observation leads to a crucial insight: the sequential deliberation phase does not necessarily demand peak intrinsic reasoning power from the model. Instead, it relies more heavily on the model’s ability to perform comprehensive analysis, synthesis, and summarization of the thought traces generated in the first stage. This suggests that the heavy thinking paradigm can be effectively scaled by utilizing larger, more instruction-following models for deliberation, even if their specialized reasoning capabilities are not the primary driver of success.

4.3. Effectiveness of Iteration Deliberation

In our heavy thinking framework, we introduce an iterative deliberation mechanism to allow the model to recursively analyze parallel trajectories by incorporating previously generated summary information. In our experiments, we fix the number of outputs for both parallel reasoning and sequential deliberation at $K = K^{(1)} = \dots = K^{(N)} = 8$, and N is set as 4. To evaluate the robustness and scalability of this approach, we conduct experiments across three models of varying scales, i.e., R1-Distill-Qwen-7B, R1-Distill-Qwen3-8B, and DeepSeek-R1-0528.

As illustrate in Figure 4, we observe a consistent upward

²From the official result in Qwen3 technique report, Qwen2.5-32B-Instruct achieves 12.8% accuracy on AIME25, which is lower than R1-Distill-Qwen-7.

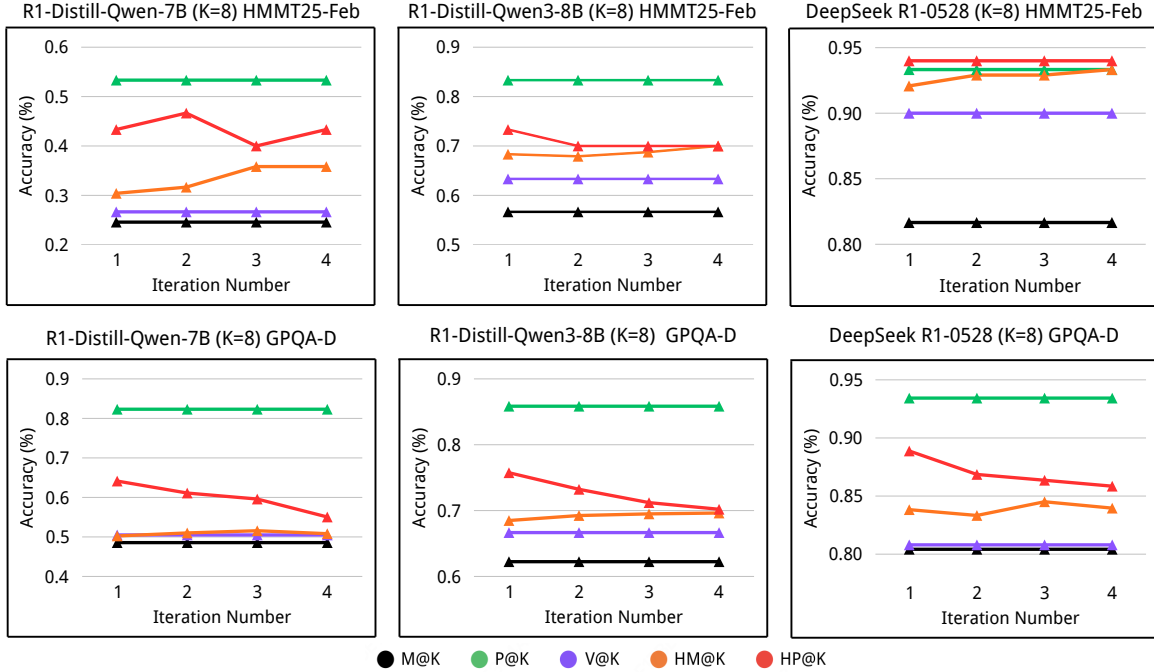


Figure 4. The effectiveness of different numbers of iterations.

trend in the HM@K metric as the number of iterations increases. This phenomenon suggests that the heavy thinking framework exhibits intrinsic scaling capabilities, where extended deliberation cycles contribute to improved collective reasoning performance. However, this gain is accompanied by a significant degradation in the HP@K metric. This divergence indicates that while iterative processing helps in certain dimensions, subsequent deliberation steps may be susceptible to interference from information generated in earlier stages. Such interference likely introduces cumulative noise or biases that constrain the model’s refinement space, ultimately limiting the potential for further performance gains. These findings highlight a critical trade-off between iterative depth and information consistency within the deliberation process.

4.4. Adaptation to Agentic Tool Use

To further explore the generalizability of the heavy thinking framework, we investigate its performance in scenarios requiring external tool interactions. Specifically, we generate reasoning trajectories that incorporate tool calls during the parallel reasoning phase. For this experiment, we select three models equipped with native tool-calling capabilities: Qwen3-8B, Qwen3-32B, and GPT-OSS-20B. We utilize a Python interpreter to provide execution feedback, which serves as a crucial signal for the deliberation process. The maximum number of interaction rounds between the model and the interpreter is capped at 50 to ensure a balance between reasoning depth and computational efficiency.

Table 3 summarizes the comparative results on the AIME25 and HMMT25 benchmarks. The empirical evidence demonstrates that the heavy thinking mode (represented by HM@4) consistently surpasses the performance of the traditional majority voting baseline (V@4) across all tested models and datasets. For instance, on the AIME25 benchmark, GPT-OSS-20B achieves an accuracy of 90.0% under the heavy thinking framework, significantly exceeding the 83.3% achieved by voting. These results indicate that the sequential deliberation mechanism effectively leverages the feedback signals from tool execution, allowing the model to refine its trajectories more accurately.

5. Related Work

5.1. Parallel Reasoning in LLM

Parallel thinking has recently emerged as a vibrant research frontier, which can be viewed as a efficient but effect test-time scaling technique (Pan et al., 2025; Liu et al., 2024; Jin et al.; Rodionov et al., 2025; Yang et al., 2025b;c). Predominant among current approaches are brute-force strategies that either spawn independent trajectories with end-stage aggregation (Brown et al., 2024; Zheng et al., 2025b; Wen et al., 2025) or synchronize thoughts at rigid, pre-defined intervals (Hsu et al., 2025; Macfarlane et al., 2025). Such paradigms inherently lack adaptivity, as their branching and merging points are dictated by static schedules rather than the evolving intermediate progress of the reasoning process. While frameworks like Monte Carlo Tree Search (Zhang

Models	AIME25				HMMT25			
	M@k	P@k	V@4	HM@4	M@k	P@k	V@4	HM@4
Qwen3-8B	55.7	83.3	68.3	76.7	38.0	70.7	54.1	69.3
Qwen3-32B	63.0	89.0	83.3	80.0	40.3	81.7	63.3	68.5
GPT-OSS-20B	69.8	95.0	83.3	90.0	55.3	93.3	73.3	85.7

Table 3. Overview performance of heavy mode on tool-interleave reasoning scenarios (Heavy Mean@4, simplify as HM@4) compared to basic TTS metrics, i.e., Mean@K (M@K), Pass@K (P@K), and Vote@K (V@K).

et al., 2024a) and Tree of Thoughts (Yao et al., 2023) provide more granular parallelization, they remain tethered to hand-crafted heuristics and external verifiers. In contrast, the recently emerging heavy thinking approach offers more flexibility in implementing test time scaling, as demonstrated in Gemini (DeepMind, 2025) and Kimi K2 (Bai et al., 2025), and PaCoRe (StepFun-AI, 2025). This paper delves into the specific implementation of the heavy thinking pattern and analyzes its effectiveness across multiple tasks in various domains.

5.2. Test-time Scaling in LLM Post-training

With the development of OpenAI’s o1 (Jaech et al., 2024), DeepSeek R1 (Guo et al., 2025), and Gemini (DeepMind, 2025), the post-training with test-time scaling has been powerful and versatile techniques in LLM reasoning. These approaches aim to develop the LLM with capabilities for correcting, reflecting, critiquing, and verifying by themselves, typically increasing inference computation by extending the model’s reasoning chains through long CoT (Wei et al., 2022; Zhang et al., 2024b; Wang et al., 2025b; Zelikman et al., 2024; Lightman et al., 2024; Wang et al., 2025a; 2024a). Recently, multiple efforts devote to improve the reasoning capabilities by introducing efficiency and stability RLVR, which aims to optimize LLM via outcome-based, automatically checkable rewards, or step-wise signals (Yu et al., 2025; Yue et al., 2025; Liu et al., 2025; Chen et al., 2025). In this paper, we introduce RLVR in heavy thinking to investigate whether the LLM can break the reasoning boundaries.

6. Conclusion

In this paper, we have systematically investigated heavy thinking as a novel TTS strategy to enhance the reasoning capabilities of LLMs, and distill the whole workflow into a readable skill for agentic harness. By introducing a framework centered on parallel reasoning and sequential deliberation, we provided a clear structural understanding of

how test-time computation translates into superior task performance. Our extensive evaluations reveal that heavy thinking consistently outperforms traditional Best-of-N strategies (e.g., voting method), particularly in models with high intrinsic reasoning potential where performance can approach the theoretical Pass@K limit. Extensive detailed analysis is also conducted to show the effect of the heavy thinking. Most importantly, our findings demonstrate that RLVR can also substantially improve the model’s reasoning capabilities. In the future, we aim to conduct a more granular analysis of the performance of heavy thinking trajectories within RL frameworks.

References

- Bai, Y., Bao, Y., Chen, G., Chen, J., Chen, N., Chen, R., Chen, Y., Chen, Y., Chen, Y., Chen, Z., Cui, J., Ding, H., Dong, M., Du, A., Du, C., Du, D., Du, Y., Fan, Y., Feng, Y., Fu, K., Gao, B., Gao, H., Gao, P., Gao, T., Gu, X., Guan, L., Guo, H., Guo, J., Hu, H., Hao, X., He, T., He, W., He, W., Hong, C., Hu, Y., Hu, Z., Huang, W., Huang, Z., Huang, Z., Jiang, T., Jiang, Z., Jin, X., Kang, Y., Lai, G., Li, C., Li, F., Li, H., Li, M., Li, W., Li, Y., Li, Y., Li, Z., Li, Z., Lin, H., Lin, X., Lin, Z., Liu, C., Liu, C., Liu, H., Liu, J., Liu, J., Liu, L., Liu, S., Liu, T. Y., Liu, T., Liu, W., Liu, Y., Liu, Y., Liu, Y., Liu, Y., Liu, Z., Lu, E., Lu, L., Ma, S., Ma, X., Ma, Y., Mao, S., Mei, J., Men, X., Miao, Y., Pan, S., Peng, Y., Qin, R., Qu, B., Shang, Z., Shi, L., Shi, S., Song, F., Su, J., Su, Z., Sun, X., Sung, F., Tang, H., Tao, J., Teng, Q., Wang, C., Wang, D., Wang, F., and Wang, H. Kimi K2: open agentic intelligence. *CoRR*, abs/2507.20534, 2025.
- Brown, B. C. A., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. Large language monkeys: Scaling inference compute with repeated sampling. *CoRR*, abs/2407.21787, 2024.
- Chen, A., Li, A., Gong, B., Jiang, B., Fei, B., Yang, B., Shan, B., Yu, C., Wang, C., Zhu, C., Xiao, C., Du, C., Zhang, C., Qiao, C., Zhang, C., Du, C., Guo, C., Chen, D., Ding, D., Sun, D., Li, D., Jiao, E., Zhou, H., Zhang, H., Ding, H., Sun, H., Feng, H., Cai, H., Zhu, H., Sun, J., Zhuang, J., Cai, J., Song, J., Zhu, J., Li, J., Tian, J., Liu, J., Xu, J., Yan, J., Liu, J., He, J., Feng, K., Yang, K., Xiao, K., Han, L., Wang, L., Yu, L., Feng, L., Li, L., Zheng, L., Du, L., Yang, L., Zeng, L., Yu, M., Tao, M., Chi, M., Zhang, M., Lin, M., Hu, N., Di, N., Gao, P., Li, P., Zhao, P., Ren, Q., Xu, Q., Li, Q., Wang, Q., Tian, R., Leng, R., Chen, S., Chen, S., Shi, S., Weng, S., Guan, S., Yu, S., Li, S., Zhu, S., Li, T., Cai, T., Liang, T., Cheng, W., Kong, W., Li, W., Chen, X., Song, X., Luo, X., Su, X., Li, X., Han, X., Hou, X., Lu, X., Zou, X., Shen, X., Gong, Y., Ma, Y., Wang, Y., Shi, Y., Zhong, Y., and Duan,

- Y. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *CoRR*, abs/2506.13585, 2025.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021.
- Claude. The claude 3 model family: Opus, sonnet, haiku. URL <https://api.semanticscholar.org/CorpusID:270640496>.
- DeepMind, G. Advanced version of gemini with deep reasoning officially achieves gold medal standard at the international mathematical olympiad, July 2025. Accessed: 2025-07-30.
- DeepSeek-AI, Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., Lu, C., Zhao, C., Deng, C., Xu, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Li, E., Zhou, F., Lin, F., Dai, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Li, H., Liang, H., Wei, H., Zhang, H., Luo, H., Ji, H., Ding, H., Tang, H., Cao, H., Gao, H., Qu, H., Zeng, H., Huang, J., Li, J., Xu, J., Hu, J., Chen, J., Xiang, J., Yuan, J., Cheng, J., Zhu, J., Ran, J., Jiang, J., Qiu, J., Li, J., Song, J., Dong, K., Gao, K., Guan, K., Huang, K., Zhou, K., Huang, K., Yu, K., Wang, L., Zhang, L., Wang, L., Zhao, L., Yin, L., Guo, L., Luo, L., Ma, L., Wang, L., Zhang, L., Di, M. S., Xu, M. Y., Zhang, M., Zhang, M., Tang, M., Zhou, M., Huang, P., Cong, P., Wang, P., Wang, Q., Zhu, Q., Li, Q., Chen, Q., Du, Q., Xu, R., Ge, R., Zhang, R., Pan, R., Wang, R., Yin, R., Xu, R., Shen, R., Zhang, R., Liu, S. H., Lu, S., Zhou, S., Chen, S., Cai, S., Chen, S., Hu, S., Liu, S., Hu, S., Ma, S., Wang, S., Yu, S., Zhou, S., Pan, S., Zhou, S., Ni, T., Yun, T., Pei, T., Ye, T., Yue, T., Zeng, W., Liu, W., Liang, W., Pang, W., Luo, W., Gao, W., Zhang, W., Gao, X., Wang, X., Bi, X., Liu, X., Wang, X., Chen, X., Zhang, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Li, X., Yang, X., Li, X., Chen, X., Su, X., Pan, X., Lin, X., Fu, X., Wang, Y. Q., Zhang, Y., Xu, Y., Ma, Y., Li, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Qian, Y., Yu, Y., Zhang, Y., Ding, Y., Shi, Y., Xiong, Y., He, Y., Zhou, Y., Zhong, Y., Piao, Y., Wang, Y., Chen, Y., Tan, Y., Wei, Y., Ma, Y., Liu, Y., Yang, Y., Guo, Y., Wu, Y., Wu, Y., Cheng, Y., Ou, Y., Xu, Y., Wang, Y., Gong, Y., Wu, Y., Zou, Y., Li, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Zhao, Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Huang, Z., Wu, Z., Li, Z., Zhang, Z., Xu, Z., Wang, Z., Gu, Z., Zhu, Z., Li, Z., Zhang, Z., Xie, Z., Gao, Z., Pan, Z., Yao, Z., Feng, B., Li, H., Cai, J. L., Ni, J., Xu, L., Li, M., Tian, N., Chen, R. J., Jin, R. L., Li, S. S., Zhou, S., Sun, T., Li, X. Q., Jin, X., Shen, X., Chen, X., Song, X., Zhou, X., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Huang, Z., Xu, Z., Zhang, Z., Ji, D., Liang, J., Guo, J., Chen, J., Xia, L., Wang, M., Li, M., Zhang, P., Chen, R., Sun, S., Wu, S., Ye, S., Wang, T., Xiao, W. L., An, W., Wang, X., Sun, X., Wang, X., Tang, Y., Zha, Y., Zhang, Z., Ju, Z., Zhang, Z., and Qu, Z. Deepseek-v3.2: Pushing the frontier of open large language models, 2025.
- Gao, H., Geng, J., Hua, W., Hu, M., Juan, X., Liu, H., Liu, S., Qiu, J., Qi, X., Wu, Y., Wang, H., Xiao, H., Zhou, Y., Zhang, S., Zhang, J., Xiang, J., Fang, Y., Zhao, Q., Liu, D., Ren, Q., Qian, C., Wang, Z., Hu, M., Wang, H., Wu, Q., Ji, H., and Wang, M. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence, 2026. URL <https://arxiv.org/abs/2507.21046>.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Ding, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Chen, J., Yuan, J., Tu, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., You, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Zhou, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y.,

- Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning, 2025.
- Hermes-Agent. Hermes-agent. <https://github.com/nousresearch/hermes-agent>, 2024.
- Hsu, C., Buffelli, D., McGowan, J., Liao, F., Chen, Y., Vakili, S., and Shiu, D. Group think: Multiple concurrent reasoning agents collaborating at token level granularity. *CoRR*, abs/2505.11107, 2025.
- Jaech, A., Kalai, A., Lerer, A., Richardson, A., El-Kishky, A., Low, A., Helyar, A., Madry, A., Beutel, A., Carney, A., Iftimie, A., Karpenko, A., Passos, A. T., Neitz, A., Prokofiev, A., Wei, A., Tam, A., Bennett, A., Kumar, A., Saraiva, A., Vallone, A., Duberstein, A., Kondrich, A., Mishchenko, A., Applebaum, A., Jiang, A., Nair, A., Zoph, B., Ghorbani, B., Rossen, B., Sokolowsky, B., Barak, B., McGrew, B., Minaiev, B., Hao, B., Baker, B., Houghton, B., McKinzie, B., Eastman, B., Lugaresi, C., Bassin, C., Hudson, C., Li, C. M., de Bourcy, C., Voss, C., Shen, C., Zhang, C., Koch, C., Orsinger, C., Hesse, C., Fischer, C., Chan, C., Roberts, D., Kappler, D., Levy, D., Selsam, D., Dohan, D., Farhi, D., Mely, D., Robinson, D., Tsipras, D., Li, D., Oprica, D., Freeman, E., Zhang, E., Wong, E., Proehl, E., Cheung, E., Mitchell, E., Wallace, E., Ritter, E., Mays, E., Wang, F., Such, F. P., Raso, F., Leoni, F., Tsimpouras, F., Song, F., von Lohmann, F., Sulit, F., Salmon, G., Parascandolo, G., Chabot, G., Zhao, G., Brockman, G., Leclerc, G., Salman, H., Bao, H., Sheng, H., Andrin, H., Bagherinezhad, H., Ren, H., Lightman, H., Chung, H. W., Kivlichan, I., O’Connell, I., Osband, I., Gilaberte, I. C., and Akkaya, I. Openai o1 system card. *CoRR*, abs/2412.16720, 2024.
- Jin, T., Cheng, E. Y., Ankner, Z., Saunshi, N., Elias, B. M., Yazdanbakhsh, A., Ragan-Kelley, J., Subramanian, S., and Carbin, M. Learning to keep a promise: Scaling language model decoding parallelism with learned asynchronous decoding. In *ICML*. OpenReview.net.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *ICLR*. OpenReview.net, 2024.
- Liu, B., Guertler, L., Yu, S., Liu, Z., Qi, P., Balcells, D., Liu, M., Tan, C., Shi, W., Lin, M., Lee, W. S., and Jaques, N. SPIRAL: self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *CoRR*, abs/2506.24119, 2025.
- Liu, M., Zeng, A., Wang, B., Zhang, P., Tang, J., and Dong, Y. APAR: llms can do auto-parallel auto-regressive decoding. *CoRR*, abs/2401.06761, 2024.
- Macfarlane, M., Kim, M., Jojic, N., Xu, W., Caccia, L., Yuan, X., Zhao, W., Shi, Z., and Sordoni, A. Instilling parallel reasoning into language models. *ICML 2025*, 2025.
- Meng, Q., Wang, Y., Chen, L., Li, Y., Wu, W., Jiang, W., Wang, Q., Lu, C., Gao, Y., Wu, Y., and Hu, Y. Agent harness for large language model agents: A survey. *Preprints*, April 2026. doi: 10.20944/preprints202604.0428.v3.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. *CoRR*, abs/2508.10925, 2025a.
- OpenAI. Introducing gpt-5, 2025b. URL <https://openai.com/index/introducing-gpt-5/>.
- OpenClaw. Openclaw. <https://github.com/openclaw/openclaw>, 2024.
- Pan, J., Li, X., Lian, L., Snell, C., Zhou, Y., Yala, A., Darrell, T., Keutzer, K., and Suhr, A. Learning adaptive parallel reasoning with language models. *CoRR*, abs/2504.15466, 2025.
- Rodionov, G., Garipov, R., Shutova, A., Yakushev, G., Egiazarian, V., Sinitsin, A., Kuznedelev, D., and Alistarh, D. Hogwild! inference: Parallel LLM generation via concurrent attention. *CoRR*, abs/2504.06261, 2025.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient RLHF framework. In *EuroSys*, pp. 1279–1297. ACM, 2025.
- StepFun-AI. Pacore: Learning to scale test-time compute with parallel coordinated reasoning. <https://github.com/stepfun-ai/PaCoRe>, 2025. Accessed: 2025.
- Wang, J., Sun, Q., Li, X., and Gao, M. Boosting language models reasoning with chain-of-knowledge prompting. In Ku, L., Martins, A., and Srikumar, V. (eds.), *ACL*, pp. 4958–4981. Association for Computational Linguistics, 2024a.
- Wang, J., Jiang, J., Liu, Y., Zhang, M., and Cai, X. Prejudge-before-think: Enhancing large language models at test-time by process prejudice reasoning. *CoRR*, abs/2504.13500, 2025a.
- Wang, J., Zhou, Y., Zhang, X., Bao, M., and Yan, P. Self-evolutionary large language models through uncertainty-enhanced preference optimization. In *AAAI*, volume 39, pp. 25362–25370, 2025b.

- Wang, J., Zhang, J., Guo, Q., Guo, L., Li, R., Zhang, C., Peng, C., Wang, C., Zhao, D., Shi, J., Wang, J., Feng, L., Shen, M., Li, Q., An, S., Wang, S., Shi, W., Xi, X., Li, X., Cao, X., Lu, Y., Zhao, Y., Chen, Z., Lin, Z., Wang, W., Pei, P., and Cai, X. Longcat-flash-prover: Advancing native formal reasoning via agentic tool-integrated reinforcement learning, 2026.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024b.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.
- Wen, H., Su, Y., Zhang, F., Liu, Y., Liu, Y., Zhang, Y., and Li, Y. Parathinker: Native parallel thinking as a new paradigm to scale LLM test-time compute. *CoRR*, abs/2509.04475, 2025.
- Xu, R. and Yan, Y. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025a.
- Yang, X., An, Y., Liu, H., Chen, T., and Chen, B. Multiverse: Your language models secretly decide how to parallelize and merge generation. *CoRR*, abs/2506.09991, 2025b.
- Yang, X., Chen, T., and Chen, B. APE: faster and longer context-augmented generation via adaptive parallel encoding. In *ICLR*. OpenReview.net, 2025c.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *NeurIPS*, 2023.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Dai, W., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W., Zhang, Y., Yan, L., Qiao, M., Wu, Y., and Wang, M. DAPO: an open-source LLM reinforcement learning system at scale. *CoRR*, abs/2503.14476, 2025.
- Yue, Y., Yuan, Y., Yu, Q., Zuo, X., Zhu, R., Xu, W., Chen, J., Wang, C., Fan, T., Du, Z., Wei, X., Yu, X., Liu, G., Liu, J., Liu, L., Lin, H., Lin, Z., Ma, B., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhang, R., Liu, X., Wang, M., Wu, Y., and Yan, L. VAPO: efficient and reliable reinforcement learning for advanced reasoning tasks. *CoRR*, abs/2504.05118, 2025.
- Zelikman, E., Harik, G., Shao, Y., Jayasiri, V., Haber, N., and Goodman, N. D. Quiet-star: Language models can teach themselves to think before speaking. *CoRR*, abs/2403.09629, 2024.
- Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., Xie, C., Wang, C., Yin, D., Zeng, H., Zhang, J., Wang, K., Zhong, L., Liu, M., Lu, R., Cao, S., Zhang, X., Huang, X., Wei, Y., Cheng, Y., An, Y., Niu, Y., Wen, Y., Bai, Y., Du, Z., Wang, Z., Zhu, Z., Zhang, B., Wen, B., Wu, B., Xu, B., Huang, C., Zhao, C., Cai, C., Yu, C., Li, C., Ge, C., Huang, C., Zhang, C., Xu, C., Zhu, C., Li, C., Yin, C., Lin, D., Yang, D., Jiang, D., Ai, D., Zhu, E., Wang, F., Pan, G., Wang, G., Sun, H., Li, H., Li, H., Hu, H., Zhang, H., Peng, H., Tai, H., Zhang, H., Wang, H., Yang, H., Liu, H., Zhao, H., Liu, H., Yan, H., Liu, H., Chen, H., Li, J., Zhao, J., Ren, J., Jiao, J., Zhao, J., Yan, J., Wang, J., Gui, J., Zhao, J., Liu, J., Li, J., Li, J., Lu, J., Wang, J., Yuan, J., Li, J., Du, J., Du, J., Liu, J., Zhi, J., Gao, J., Wang, K., Yang, L., Xu, L., Fan, L., Wu, L., Ding, L., Wang, L., Zhang, M., Li, M., Xu, M., Zhao, M., and Zhai, M. GLM-4.5: agentic, reasoning, and coding (ARC) foundation models. *CoRR*, abs/2508.06471, 2025.
- Zhang, D., Huang, X., Zhou, D., Li, Y., and Ouyang, W. Accessing GPT-4 level mathematical olympiad solutions via monte carlo tree self-refine with llama-3 8b. *CoRR*, abs/2406.07394, 2024a.
- Zhang, D., Zhou, S., Hu, Z., Yue, Y., Dong, Y., and Tang, J. Rest-mcts*: LLM self-training via process reward guided tree search. In *NeurIPS*, 2024b.
- Zheng, C., Liu, S., Li, M., Chen, X., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., and Lin, J. Group sequence policy optimization. *CoRR*, abs/2507.18071, 2025a.
- Zheng, T., Zhang, H., Yu, W., Wang, X., Dai, R., Liu, R., Bao, H., Huang, C., Huang, H., and Yu, D. Parallel-r1: Towards parallel thinking via reinforcement learning. *CoRR*, abs/2509.07980, 2025b.

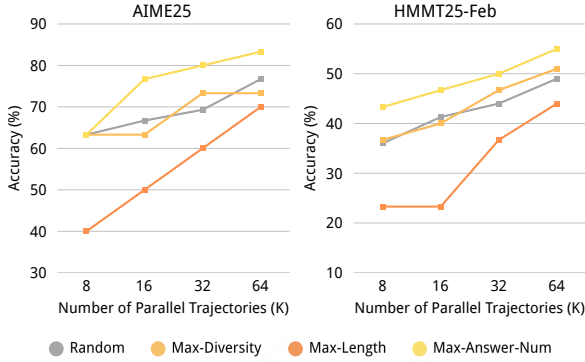


Figure 5. When choosing different permutations. Random: randomly select K trajectories; Max-Diversity: select K trajectories that have the highest diversity; Max-Length: select the top K trajectories based on the length; Max-Answer-Num: select the trajectories that have the highest frequency answer.

A. Impact of Parallel Trajectories

In this section, we aim to investigate the impact of the permutations of parallel reasoning. We select R1-Distill-Qwen3-8B model as the basic backbone, and evaluate across two challenging benchmarks, including AIME25 and HMMT25-Feb. For each problem, we generate 256 parallel trajectories, and compare four distinct trajectory selection (permutation) strategies: Random, Max-Diversity, Max-Length, and Max-Answer-Num.

The experimental results are illustrated in Figure 5, we draw the following suggestions. 1) Consistent upward trend in accuracy for all selection methods as the number of parallel trajectories K increases. This demonstrates that increasing the inference budget through parallelization consistently benefits model performance, regardless of the selection heuristic employed. 2) Interestingly, the Max-Diversity strategy yields performance comparable to Random sampling. This suggests that while diversity is a natural byproduct of increased sampling temperatures, explicitly optimizing for trajectory diversity does not provide a significant marginal gain. Conversely, the Max-Length strategy performs the worst among all tested methods. This indicates that a preference for longer outputs introduces substantial noise, suggesting that verbosity does not necessarily correlate with reasoning quality in parallel inference settings. 3) The Max-Answer-Num strategy significantly outperforms all other baselines. This result highlights that leveraging consensus (majority voting) to select parallel trajectories effectively increases the proportion of correct reasoning paths within the candidate set. Consequently, this provides a more robust foundation for the heavy thinking module to perform synthesis, summarization, and final logical deduction.

B. Advancing Heavy Thinking via RLVR

Through extensive preliminary experiments, we observe that under the "heavy thinking" mode, the HP@ K metric can significantly outperform HM@ K in specific scenarios. This observation motivates a critical research question: can applying Reinforcement Learning from Verifiable Rewards (RLVR) directly to heavy thinking trajectories further elevate the upper bound of the model's reasoning capabilities?

To investigate this, we reuse the parallel reasoning trajectories generated in Experiment 3. We specifically select queries with a pass rate in the range $[0, 0.625]$ and randomly sample $K \in \{8, 16\}$ trajectories to be encapsulated as serialized memory caches. Our experimental framework is built upon VeRL (Sheng et al., 2025), employing GSPO (Zheng et al., 2025a) as the reinforcement learning algorithm. We utilize R1-Distill-Qwen-7B as the backbone model.

The experimental results are illustrated in Figure 6. During the initial training phase (within the first 100 steps), the model exhibits a consistent growth trend on both the training and test sets. Notably, the HM@4 metric achieves a further improvement of approximately 10%. However, we observe a distinct divergence in stability between the two configurations. For the $K = 16$ group, the model experiences a significant entropy collapse after 100 steps. In contrast, the $K = 8$ configuration demonstrates superior stability throughout the training process. We think that this instability in the $K = 16$ setting is primarily attributed to the maximum sequence length limitations of the R1-Distill-Qwen-7B model, which may lead to truncated or suboptimal training signals when handling longer serialized contexts. While the initial results are promising, the dynamics of heavy thinking under RLVR require further exploration.

C. Prompt and Skill for Heavy Thinking

The prompt for memory cache is shown in Figure 7.

The skill file for agentic harness is shown in Figure 8, Figure 9, and Figure 10.



Figure 6. The RL training on heavy thinking. Blue curve denotes the number of parallel trajectories is 8, while the green curve denotes 16. We use the VeRL framework to perform RLVR.

Prompt

You are a great reasoner.

Here is a problem, and multiple thinkers attempt to give their thought processes independently.

Each thinker has written its own thought process towards the final answer. Each thinker is encouraged to take the other thinkers' progress into account to reach the final answer.

===== Problem =====

{problem}

===== Problem End =====

===== Thinkers Thought Process =====

{parallel_trajectories_prompt}

===== Thinkers Thought Process End =====

Look at the above problem and thought process from each thinker, summarize from these thought processes and finally give your answer.

Summarize their thinking on the problem and try to summarize the thinking of these thinkers. Analyze the differences in thinking between these thinkers and try to analyze which thought process is correct.

Note:

- It is generally believed that when most thinkers get the same answer, the answer may be correct. But you can't do it so superficially, because the correct answer may come from very few thinkers, or even no thinker gives the correct answer. For this reason, when you summarizing, you NEED adhere to the principles of professionalism and critical thinking, carefully identify these thought processes, and give a summary and final answer.
 - If you realize that none of these thinkers have answered correctly, you can even learn from the wrong experiences in the thought process of these thinkers and re-think the given problem to give the answer you think is most correct.
 - Please DO NOT just solve the given problem independently like other thinkers, but summarize the thought process of all thinkers. In other words, you need to give the summary first, and then give the final answer, you can re-think this problem only if you realize that none of these thinkers have answered correctly.
- For the output format of your final answer after the summary, you should follow the similar format with thinkers:
- If the problem is mathematics or STEM and most thinkers have put the answer within boxed, your final answer after the summary should within boxed too.
 - If the problem is code contest and most thinkers have put the result code within a block like `````, your final answer should also within the block.

Parallel Trajectories Prompting Function

```
response_prompt = ""
for ei, response in enumerate(parallel_reason_content_list):
    response_prompt += "# ----- Thinker #{i} ----- \n\n{response}\n\n".format(ei + 1, response)
prompt = prompt.replace("{problem}", query).replace("{response_prompt}", response_prompt)
```

Figure 7. The prompt of heavy thinking in serialized memory cache.

HeavySkill.md

```

# HeavySkill: Heavy Thinking
## Overview
HeavySkill is a reasoning amplification technique that decomposes complex problem-solving into two stages:
1. Parallel Reasoning — Generate multiple independent reasoning trajectories for the same problem
2. Sequential Deliberation — Synthesize all trajectories through critical analysis to produce a superior final answer
This skill should be activated when facing complex reasoning tasks where a single chain-of-thought may be insufficient.

## When to Activate
Activate HeavySkill when the task involves:
- Mathematical reasoning (competition math, STEM problems)
- Complex logical deduction
- Code competition / algorithmic problems
- Tasks where correctness is critical and verifiable
- Problems where you are uncertain about your initial approach
Do NOT activate for:
- Simple factual questions
- Casual conversation
- Straightforward code edits with obvious solutions
- Tasks that are primarily about information retrieval
## Execution Protocol
### Stage 1: Parallel Reasoning
Spawn K independent reasoning agents (recommended K=3~5 in harness, K=8+ in workflow) to solve the same problem. Each agent must reason completely independently without seeing others' work.
Instructions for each parallel agent:
- Solve the given problem step by step from scratch
- Show complete reasoning chain
- Arrive at a final answer
- Do NOT communicate with other agents
- Use different reasoning approaches when possible (e.g., algebraic vs. geometric, brute force vs. elegant)
Agent spawn prompt template:
...
Solve the following problem step by step. Show your complete reasoning and arrive at a final answer.
Problem: {query}
Think carefully and solve this independently. Show all work.
...

(...)

```

Figure 8. The skill file of heavy thinking (part I).

HeavySkill.md

```
(...)
### Stage 2: Sequential Deliberation
After collecting all K trajectories, perform a meta-analysis:
1. Identify answer distribution — What answers appear and how frequently?
2. Analyze reasoning quality — Which chains are logically sound vs. flawed?
3. Cross-validate — Do different approaches confirm the same result?
4. Critical evaluation — Apply professional skepticism:
- Majority consensus is a signal but NOT proof of correctness
- A minority answer backed by rigorous logic may be correct
- All trajectories may be wrong — be prepared to reason anew
5. Synthesize final answer — Produce the best answer based on analysis
Deliberation prompt framework:
...
Multiple independent thinkers have attempted this problem. Analyze their reasoning:
Problem: {query}
Thinker #1: {trajectory_1}
Thinker #2: {trajectory_2}
...
Thinker #K: {trajectory_K}
Your task:
- Analyze the thought processes of all thinkers
- Identify logical errors or gaps in each approach
- Determine which reasoning path(s) are most sound
- If all thinkers are wrong, reason independently from their mistakes
- Provide the definitive final answer
...
## Implementation in Claude Code Harness
When activated in Claude Code, execute as follows:
1. Identify the problem — Extract the core reasoning task from the user's request
2. Spawn parallel agents — Use the Agent tool to launch K=3 independent reasoning agents in a single message (parallel execution)
3. Collect results — Wait for all agents to complete and gather their outputs
4. Deliberate — Perform the sequential deliberation analysis yourself (do NOT delegate this step)
5. Output — Provide the final synthesized answer to the user
(...)

```

Figure 9. The skill file of heavy thinking (part II).

HeavySkill.md

(...)

Key Principles

- **Independence is critical** — Parallel agents must not share context or see each other's work
- **Diversity helps** — Encourage different problem-solving strategies across agents
- **Deliberation is synthesis, not voting** — Don't just pick the majority answer; analyze reasoning quality
- **Language consistency** — Match the language of the final output to the user's query language
- **Format consistency** — Match output format to what the task expects (boxed answers for math, code blocks for programming, etc.)

Iterative Refinement (Optional)

For extremely challenging problems, iterate:

1. Run Stage 1 + Stage 2 as above
2. Feed the deliberation result back as an additional "expert thinker" trajectory
3. Re-run Stage 2 with the augmented trajectory set
4. Repeat until convergence (typically 2-3 iterations max)

Output Format

Your final output should:

- Present **ONLY** the final answer (not the meta-analysis)
- Follow the format conventions of the domain:
 - Math/STEM: answer in `\boxed{}`
 - Code: solution in a code block
 - General: clean prose response
- Match the language of the original query

Figure 10. The skill file of heavy thinking (part III).